

Aspect	Text Blocks (Preview) — Java 13
Java Version	Java 13
Year	2019
Feature	Text Blocks (Preview)
UniqueClassName	TextBlocks
What it is	A multi-line string literal using <code>"""</code> that makes it easier to write and maintain multi-line text directly in Java source code.
Main purpose	Improve readability and maintainability of large, multi-line strings such as JSON, XML, SQL, HTML, and formatted messages.
Syntax	<code>String json = """{"name":"John"}""";</code>
Example	<pre>java
String html = """<html><body><h1>Hello</h1></body></html>""";
</pre>
Advantages	1. Much cleaner multi-line strings. 2. Reduces <code>\n</code> and escaped quotes. 3. Improves readability of embedded JSON/XML/SQL/HTML. 4. Makes source code closer to the actual text being represented. 5. Easier maintenance when large text changes.
Disadvantages	1. Preview feature in Java 13, so it was not yet finalized. 2. Developers needed preview-feature support to compile/run it. 3. Indentation/whitespace behavior requires understanding of text-block processing. 4. Not useful for ordinary short strings. 5. Existing applications using older Java versions cannot use it without upgrading/adapting the code.
When to use	Use when a string naturally spans multiple lines or contains many quotation marks and escape sequences.
When NOT to use	Avoid for short strings, dynamically generated content, strings that require complex formatting logic, or projects that must remain compatible with Java versions before text blocks were available.
Real-time use case 1 — JSON	Useful for embedding a JSON request/response template in tests or API clients. <pre>java
String json = """
 {
 "name": "Anurag",
 "role": "Developer"
 }
 """;
</pre>
Real-time use case 2 — SQL	Useful for readable multi-line SQL queries. <pre>java
String sql = """
 SELECT id, name, email
 FROM employees
</pre>

	WHERE department = 'IT' AND active = true ORDER BY name; """;
Real-time use case 3 — HTML	Useful for generating or testing HTML fragments.java String html = """" <div class="user"> <h2>John</h2> <p>Developer</p> </div> """";
Real-time use case 4 — XML	Useful for SOAP/XML payloads, configuration templates, and integration tests.java String xml = """" <user> <name>John</name> <role>Admin</role> </user> """";
Real-time use case 5 — Unit tests	Excellent for expected multi-line output, JSON fixtures, SQL, XML, or HTTP payloads in tests.
Performance impact	Primarily a source-code readability feature , not a performance optimization. It does not inherently make string processing faster.
Security considerations	Text blocks do not provide encryption, sanitization, validation, or injection protection. SQL/HTML/JSON values should still be safely parameterized/escaped where appropriate.
Alternative in Java 13	Traditional string literals using " with explicit \n, concatenation, or String.join().
Traditional alternative example	java String sql = "SELECT id, name, email\n" + "FROM employees\n" + "WHERE active = true\n" + "ORDER BY name";
Better modern alternative	Text blocks became a permanent Java language feature in Java 15. For modern Java applications, prefer finalized text blocks rather than treating them as a preview feature.
Best suited for	SQL, JSON, XML, HTML, Markdown, templates, test fixtures, multi-line messages, and other static multi-line content.
Not a replacement for	Template engines, resource files, external configuration, localization systems, parameterized SQL, or dynamically constructed documents.
Overall assessment	Very useful for readability and maintainability , particularly when embedding structured multi-line text. Its Java 13 form should be viewed as an early preview; for production code, use the finalized text-block feature available from Java 15 onward.

Before vs. Text Blocks

Requirement	Traditional Java String	Text Block
-------------	-------------------------	------------

Single-line string	String s = "Hello";	String s = """"Hello""""; — unnecessary
Multi-line text	Many \n or concatenations	Natural multi-line layout
JSON	Lots of escaped quotes	Much easier to read
SQL	String concatenation	Looks like SQL
XML	Escaping + concatenation	Looks like XML
HTML	Difficult to maintain	Much more readable
Large static test data	Verbose	Convenient
Dynamic text	Usually better	Usually not the primary choice
Java 13 status	Stable	Preview
Modern Java status	Still available	Finalized since Java 15

Key takeaway

Text Blocks (Preview) in Java 13 were primarily a readability/maintainability improvement, not a performance feature. They are particularly valuable when Java code contains **SQL, JSON, XML, HTML, or other multi-line structured text.**

The important evolution is:

Java 13 → Text Blocks (Preview)

Java 14 → Text Blocks (Second Preview)

Java 15 → Text Blocks (Final)

So, if you're documenting Java features version-by-version, classify **Java 13 Text Blocks** specifically as a **preview feature**, while noting that the feature became permanent in **Java 15**.